Computer Organization and Architecture (CSL251)
LAB-7
2025-26 W

Total Marks: 150 (3 × 50)
Submission: Assembly source code + required screenshots

Problem 1: CSR Protection and Access Fault Demonstration

(50 points)

Show how CSR privilege protection prevents lower privilege levels from accessing restricted registers.

Program Requirements

1. Start execution in Machine mode.
2. Set mtvec to a machine trap handler.
3. Configure mstatus and mepc to enter User mode using mret.
4. In User mode:
    - Attempt to read or write a machine-level CSR (e.g., mtvec or mstatus).
5. This must generate an illegal instruction exception.
6. Trap handler must:
    - Read mcause, mepc, and mtval.
    - Detect CSR access violation.
    - Advance mepc to skip the instruction.
7. Resume User mode execution safely.

Observations Required

- mcause value indicating illegal instruction
- mtval showing offending instruction encoding

Problem 2: Nested Trap Handling and Interrupt Masking

(50 points)

Demonstrate how RISC-V handles nested traps and how interrupts can be temporarily disabled and re-enabled inside a trap handler.

Program Requirements

1. Begin execution in Machine mode.
2. Initialize the mtvec register with the address of a machine-mode trap handler.
3. Enable machine interrupts by configuring:
    - Mstatus
    - mie
4. Configure the machine timer to generate periodic interrupts.
5. When the first timer interrupt occurs:
    - Enter the trap handler.
    - Disable interrupts by clearing the MIE bit in mstatus.
6. Inside the handler:
    - Manually trigger another trap using ecall.
7. Verify:
    - The second trap does not interrupt execution while interrupts are disabled.
8. Re-enable interrupts before returning using mret.

Observations Required

- Value of mstatus.MIE before and during trap handling
- mcause values for:
    - timer interrupt
    - environment call
- Evidence that nested interrupts are controlled correctly

Problem 3: Handling of Divide-by-Zero Exception in RISC-V (50 points)

Demonstrate exception handling in RISC-V by detecting a divide-by-zero operation, recovering program execution dynamically, and maintaining exception statistics using a machine-mode trap handler.

Program Requirements

1. Begin execution in Machine mode.
2. Initialize the mtvec register with the address of a machine-mode trap handler.
3. In the main program:
    - Create a loop that performs multiple division operations.
    - Some iterations must intentionally use zero as the divisor.
4. When a divide-by-zero instruction executes:

    - An exception must occur and transfer control to the trap handler.
5. Inside the trap handler:
    - Read and store the following CSRs:
        - Mcause
        - Mepc
        - Mtval
        - mstatus
    - Verify that the exception corresponds to a divide error.
    - Maintain a counter that records how many divide-by-zero exceptions occurred.
    - Recover execution using ONE of the following strategies:
        - Replace divisor with a safe value (e.g., 1), or
        - Increment mepc to skip the faulty instruction.
6. Store the updated exception counter in memory.
7. Return to the interrupted program using mret.
8. After loop completion:
    - Store or print the total number of exceptions handled.

Observations Required

- Provide screenshots showing:
    - mcause value during exception handling
    - mepc pointing to the faulting division instruction
    - mtval contents during exception
    - Exception counter increasing after each fault
    - Program execution continuing normally after recovery

****